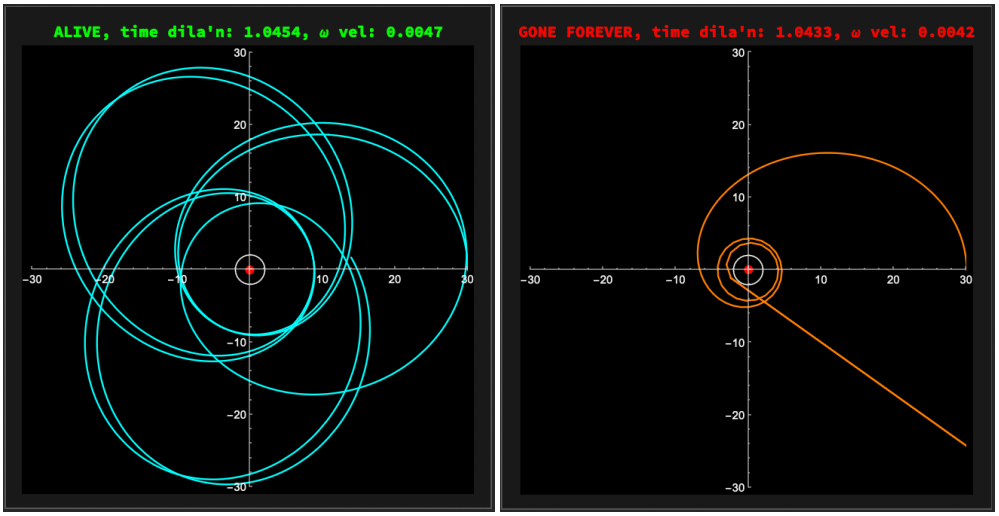
Interactive Black Hole Dynamics in $\mathcal{UD}$

Brian Beckman

Feb 2026

In this notebook, we present a “Manipulate” demonstration of orbits around a Schwarzschild black hole. The demonstration lets us explore the parameter space of a test “star” orbiting a black hole, at-will constructing stable orbits with relativistic precession or crashing the star into the event horizon, never to return.



A good way to read this notebook is to page to the end, play with the demonstration, then read the leading prose explaining the physics.

The solutions are built on the $\mathcal{UD}$ compiler proof-of-concept presented in earlier notebooks. We take advantage of the Schwarzschild metric from the Wolfram Function Repository, but compute the Christoffel symbols (force-producers), *ab-initio*, in $\mathcal{UD}$. From the Christoffel symbols, we posit the standard equations of motion for coordinates of the star in terms of proper time τ:

$$\frac{d^2 x^\mu}{d\tau^2} + \Gamma^\mu{}_{\nu\lambda} \frac{dx^\nu}{d\tau} \frac{dx^\lambda}{d\tau} = 0$$

In a future notebook, we’ll compute the Schwarzschild metric *ab-initio*, also. For now, it’s more interesting to show the visualizations.

Initialize Toolkit and Compiler

Optional: Quit for a Fresh Kernel

- *Note, if you call* **Quit** *in a notebook, evaluation may hang. If you want to "Evaluate Notebook," mark the following cell "Evaluatable" in the Cell→Property menu, call* **Quit** *once, then comment out the* **Quit** *cell, or mark it unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can "Evaluate Notebook" in a clean environment. Otherwise, you can evaluate cells in this notebook individually.*

```
(* Cell is not Evaluatable by default *)
Quit[];
```

In[1]:=
```
RTbranch = "main";
RTbase = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/" <>
    RTbranch <> "/";
RTbust = "?t=" <> ToString[Round[AbsoluteTime[]]];(*Force fresh download*)
RTrules = RTbase <> "RelativityToolkit.wl" <> RTbust;
RTSchwarzschildCompilerPoc =
  RTbase <> "APPLICATION_SCRIPTS/schwarzschild_compiler_poc.wl" <> RTbust;
Get[RTrules];
Get[RTSchwarzschildCompilerPoc];
```

» Relativity Toolkit version : 1.9.0

» Relativity Connection Symbol : Γ

Orbits

Imagine a spherical coordinate system measuring time t and spatial locations, r, θ, ϕ, stretching out from the center of a supermassive black hole to a safe distance, where spacetime is almost flat. Launch a star, as a test particle of negligible mass, to orbit the black hole. Spacetime is highly curved close to the black hole, but relativity gives us the way to figure out what our "flat" coordinates will read as we measure time and locations from our safe distance.

Integrate the equations of motion of the star as a function of its proper time. The star follows geodesics around the black hole. If the star gets too close to the event horizon at $r = 2\,M_{\text{black hole}}$ (in natural units, $G = c = 1$), the star will disappear from the Universe forever, suffering an unknowable fate, perhaps at the singularity in the center of the black hole.

Define Unknown Functions of τ

In[8]:=
```
coords = {t, r, θ, ϕ};
funcs = {t[τ], r[τ], θ[τ], ϕ[τ]};
velocities = D[funcs, τ];
```

Build ODEs from the Compiled Γ Array

The following is a straight transcription of the standard equations of motion:

$$\frac{d^2 x^\mu}{d\tau^2} + \Gamma^\mu{}_{\nu\lambda} \frac{dx^\nu}{d\tau} \frac{dx^\lambda}{d\tau} = 0 \tag{1}$$

via **myGammaArray** from the compiler script.

- *Note the resemblance to Newton's F = ma, if we interpret the term with the connection Γ as producing forces. This idea is so pervasive in all of physics that it deserves a slogan:* **Forces Come from Connections**. *We'll see it again in gauge theory in future notebooks, where the connections, instead of describing curvature of spacetime, describe curvatures in symmetry groups of gauge fields.*
- *TODO: Derive these equations as describing geodesics,* ab-initio*, from a variational principle.*

In[11]:=
```
(odes = Table[
    (D[funcs[[μ]], {τ, 2}] + Sum[
        myGammaArray[[μ, ν, λ]] *
         velocities[[ν]] * velocities[[λ]],
        (* myGammaArray indices are 1-based *)
        {ν, 1, 4}, {λ, 1, 4}] == 0) // FullSimplify,
    {μ, 1, 4}]) // TableForm
```

Out[11]//TableForm=

$$\frac{2\,M\,r'[\tau]\,t'[\tau]}{r^2 - 2\,r\,M} + t''[\tau] == 0$$

$$\frac{(r-2\,M)\,M\,t'[\tau]^2}{r^3} + r''[\tau] == \frac{M\,r'[\tau]^2}{r^2 - 2\,r\,M} + (r - 2\,M)\left(\theta'[\tau]^2 + \text{Sin}[\theta]^2\,\phi'[\tau]^2\right)$$

$$\text{Cos}[\theta]\,\text{Sin}[\theta]\,\phi'[\tau]^2 == \frac{2\,r'[\tau]\,\theta'[\tau]}{r} + \theta''[\tau]$$

$$\frac{2\,(r'[\tau] + r\,\text{Cot}[\theta]\,\theta'[\tau])\,\phi'[\tau]}{r} + \phi''[\tau] == 0$$

Substitute Unknown Functions for Formals in Γ

The $\mathcal{UD}$ proof-of-concept compiler uses formal symbols as variables. This artifact is inherited from the Wolfram Function Repository, the target "virtual machine" of the compiler, and $\mathcal{UD}$ propagates the formal symbols back up.

Map the formal symbols to our unknown functions for **NDSolve**'s sake. Set the black hole's mass, *M*, to 1, arbitrarily (the mass will have the same units as *r*, the radial distance, so about 2×10^5 solar masses if length is measured in light-seconds). **Solve** the equations algebraically into state-space form for inspection, with accelerations all to the left of equals signs.

In[12]:=
```
(odes = odes /. {r → r[τ], θ → θ[τ], ϕ → ϕ[τ], t → t[τ]}) // TableForm;
(nums = {M → 1});
(odes = Solve[odes, D[funcs, {τ, 2}]][[1]] /. {Rule → Equal} /. nums) // Column
```

Out[14]=

$$t''[\tau] == \frac{2\, r'[\tau]\, t'[\tau]}{(2-r[\tau])\, r[\tau]}$$

$$r''[\tau] == \frac{r'[\tau]^2}{-2\, r[\tau]+r[\tau]^2} - \frac{(-2+r[\tau])\, t'[\tau]^2}{r[\tau]^3} + (-2+r[\tau])\left(\theta'[\tau]^2 + \text{Sin}[\theta[\tau]]^2\, \phi'[\tau]^2\right)$$

$$\theta''[\tau] == \frac{-2\, r'[\tau]\, \theta'[\tau]+\text{Cos}[\theta[\tau]]\, r[\tau]\, \text{Sin}[\theta[\tau]]\, \phi'[\tau]^2}{r[\tau]}$$

$$\phi''[\tau] == -\frac{2\,(r'[\tau]+\text{Cot}[\theta[\tau]]\, r[\tau]\, \theta'[\tau])\, \phi'[\tau]}{r[\tau]}$$

Smoke Test

The following parameters are known empirically to produce a stable-ish orbit.

In[15]:=
```
Mval = 1;
rStart = 14;
ecc = 0.98;(* slight ellipse *)
```

Initial Angular Velocity $\dot{\phi}$

From balancing Newtonian gravitational force against centripetal force

$$\left(\frac{GMm}{r^2} = \frac{mv^2}{r}\right) \Longrightarrow \left(v = \sqrt{\frac{M}{r}}\right) \tag{2}$$

in natural units, $G = c = 1$.

- *This condition is usually good enough for a stable Newtonian orbit, but not good enough for Einstein; you can still crash a star into a black hole with this initial condition, as we see later.*

Multiply it by the eccentricity and divide again by r to get initial angular velocity $(\phi')_0$ of our elliptical orbit:

In[18]:=
```
phiVel = ecc/rStart Sqrt[Mval/rStart]
```

Out[18]=

```
0.0187083
```

Initial Coordinate Time

There is heavy time dilation near the black hole. That means that coordinate time will be highly non-linear w.r.t. τ and is subject to numerical integration. A good initial value for $dt/d\tau$, given the initial angular velocity **phiVel** above, comes from the requirement that the proper velocity, u, always have negative unit norm:

$$u.u = -1 = g_{tt}\left((t')_0^2\right) + g_{\phi\phi}\left((\phi')_0^2\right) \tag{3}$$

There are no $g_{\theta\theta}$ and g_{rr} terms because we always start with pure planar motion $(\theta')_0 = 0$ and no radial

speed $(r')_0 = 0$.

- *TODO experiment with non-zero initial radial speed in a new* **Manipulate**.

In the Schwarzschild metric, $u.u$ becomes

$$-\left(1-\frac{2M}{r_0}\right) t'_0{}^2 + r_0{}^2 (\phi')_0{}^2 = -1 \tag{4}$$

producing

$$t'_0 = \sqrt{\frac{1 + r_0{}^2 (\phi')_0{}^2}{1 - \frac{2M}{r_0}}} \tag{5}$$

In[19]:=
```
tVel = Sqrt[(1 + rStart^2 phiVel^2)/(1 - 2 Mval / rStart)]
```

Out[19]=
```
1.11656
```

Final System to Solve

In[20]:=
```
inits = {t[0] == 0, t'[0] == Echo[tVel, "initial time vel"],
    r[0] == Echo[rStart, "initial (apo/peri)apsis"], r'[0] == 0,
    θ[0] == π / 2, θ'[0] == 0, (* STRICTLY PLANAR *)
    ϕ[0] == 0, ϕ'[0] == Echo[phiVel, "initial ang vel"]};
finalSystem = Join[odes, inits];
```

» initial time vel 1.11656

» initial (apo/peri)apsis 14

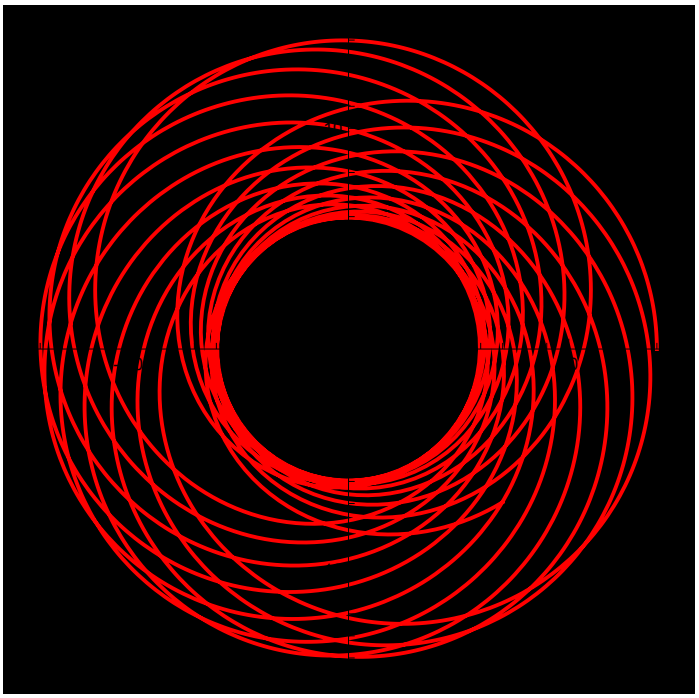
» initial ang vel 0.0187083

Run Simulation

Extreme effects of relativity are on display as precession.

In[22]:=
```
tLim = 3000;
soln = NDSolve[finalSystem, funcs, {τ, 0, tLim}];
ParametricPlot[{r[τ] Cos[ϕ[τ]], r[τ] Sin[ϕ[τ]]} /. soln,
 {τ, 0, tLim}, PlotStyle → Red, Background → Black, PlotRange → All, AspectRatio → 1]
```

Out[24]=



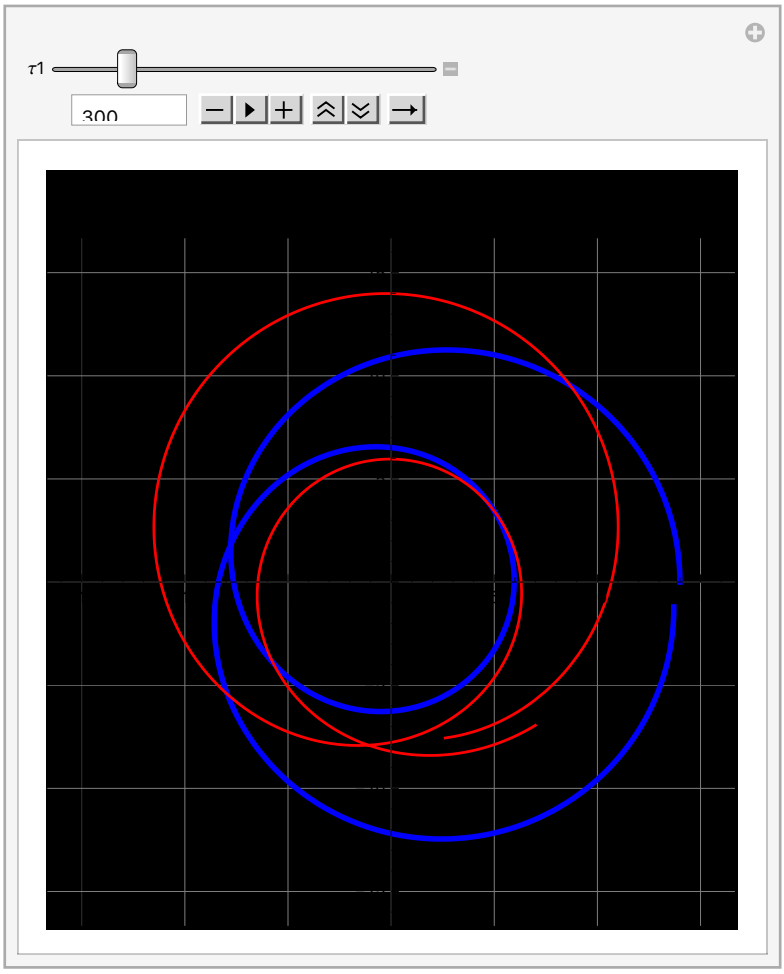
- *The 2020 Nobel Prize in Physics was awarded for measuring the mass of the supermassive black hole at the center of our Milky Way galaxy by tracking the highly elliptical orbit of the star S2. Later, measuring the orbit's precession further verified general relativity.*

Visualize Simulation Bookends

Try animating this one slowly (~12 down-speed clicks) from the initial value of $\tau = 70$.

In[25]:=
```
Manipulate[
 Show[
  ParametricPlot[{r[τ] Cos[ϕ[τ]], r[τ] Sin[ϕ[τ]]} /. soln,
   (* Beginning of Orbit: Blue *)
   {τ, 0, τ1},
   PlotStyle → {Blue, Thickness[0.008]}],
  ParametricPlot[{r[τ] Cos[ϕ[τ]], r[τ] Sin[ϕ[τ]]} /. soln,
   (* End of Orbit: Red *)
   {τ, tLim - τ1, tLim},
   PlotStyle → {Red, Thickness[0.004]} ],
  AspectRatio → 1, Axes → True, GridLines → Automatic,
  Background → Black,
  PlotRange → {{-15, 15}, {-15, 15}},
  PlotLabel →
   Style["Precession Visualized (r=14)\nBlue: Start | Red: End", 14, Bold]],
 {{τ1, tLim / 10}, 70, tLim / 2, Appearance → {"Open"}}]
```

Out[25]=



Interactive Black-Hole Lab

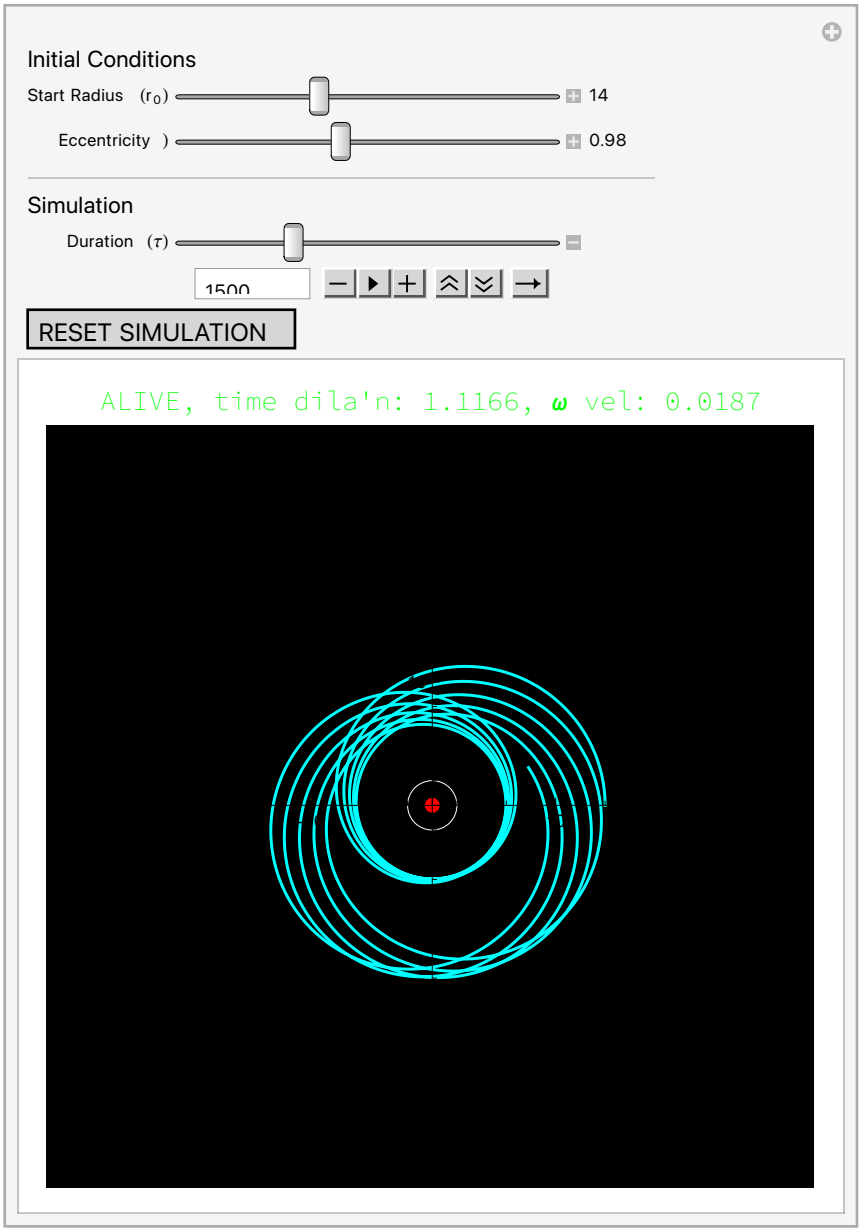
Assumes $M = 1$ and that **odes** has been defined.

In[26]:=
```
On[Assert];
Assert[ValueQ[odes]]
```

In[28]:=
```
DynamicModule[{Mval = 1,
  rMax = 30, (* still very relativistic! *)
  rCrash = 5, (* empirical *)
  rEvent = 2, (* from 1/g_rr = 1-2m/r_e == 0 *)
  rStart = 14,
  crashRad = 2.01, (* empirical: for the integrator *)

  eccMin = 0.60, (* empirical *)
  eccMax = 1.5,
  eccStart = 0.98,

  durMin = 100,
  durMax = 5000,
  durStart = 1500,

  phiVel, tVelVal, inits,
  finalSystem, soln, status, rMin
 },
 Manipulate[
  phiVel = (Sqrt[Mval / r0] * eccVel) / r0;
  tVelVal = Sqrt[(1 + r0^2 * phiVel^2) / (1 - 2 * Mval / r0)];
  inits = {
    t[0] == 0, t'[0] == tVelVal,
    r[0] == r0, r'[0] == 0,
    θ[0] == Pi / 2, θ'[0] == 0,
    ϕ[0] == 0, ϕ'[0] == phiVel};
  (* Solve with Crash Protection *)
  soln = Quiet@NDSolve[Join[odes, inits],
     funcs, {τ, 0, Tmax},
     (*Stop integration if we hit the Event Horizon (r=2)*)
     Method → {"EventLocator", "Event" → r[τ] - crashRad,
       "EventAction" → "StopIntegration"},
     MaxSteps → 100 000];
  (* Check survival *)
  rMin = Quiet@MinValue[{(r[τ] /. soln)[[1]], 0 ≤ τ ≤ Tmax}, τ];
  status = If[rMin < crashRad, "GONE FOREVER", "ALIVE"];
```

```
Column[{
  Row[Style[#, If[rMin < 2.1, Red, Green], Bold, 14] & /@
    {status,
     ", time dila'n: ", DecimalForm[tVelVal, {Infinity, 4}],
     ", ω vel: ", DecimalForm[phiVel, {Infinity, 4}]}],
  Show[
   ParametricPlot[{r[τ] Cos[ϕ[τ]], r[τ] Sin[ϕ[τ]]} /. soln, {τ, 0, Tmax},
    PlotStyle → {If[rMin < crashRad, Orange, Cyan], Thickness[0.004]},
    PlotRange → {{-rMax, rMax}, {-rMax, rMax}},
    Background → Black, ImageSize → 400],
   Graphics[{Black, Disk[{0, 0}, 2], (* Black Hole *)
     White, Circle[{0, 0}, 2], (* Event Horizon (from (1 - 2M/rEvent == 0) *)
     Red, PointSize[0.02], Point[{0, 0}] (*Singularity*)}]]},
 Alignment → Center],
Style["Initial Conditions", Bold, 12],
{{r0, rStart, "Start Radius (r₀)"}, rCrash, rMax, Appearance → "Labeled"},
{{eccVel, eccStart, "Eccentricity)"}, eccMin, eccMax, Appearance → "Labeled"},
Delimiter,
Style["Simulation", Bold, 12],
{{Tmax, durStart, "Duration (τ)"}, durMin, durMax, Appearance → {"Open"}},
Button["RESET SIMULATION",
 {r0 = rStart, eccVel = eccStart, Tmax = durStart},
 ImageSize → Medium, Method → "Queued"],
ControlPlacement → Top,
TrackedSymbols :→ {r0, eccVel, Tmax}]]
```

Out[28]=



Interesting Things to Try

Interactive Reparameterization

Start the animation of the orbit with τ at its minimum value of 100. Click the slow-button 12 times. Move the eccentricity and r_0 sliders. Watch the shape of the orbit change interactively. Try to make a stable circular orbit (my best one was at r_0 = 24, ecc = 1.0690.

- *TODO: Explain why* ecc = 1.0 *does not produce a stable circular orbit.*

- *HINT: The Newtonian velocity* $v = \sqrt{\frac{M}{r}}$ *underestimates the gravity of a black hole. The circular velocity in general relativity is greater:* $v = \sqrt{\frac{M}{r-2M}}$.
- *FOLLOW-ON QUESTION: At* $r_0 = 24$*, the ratio of Newtonian to Einsteinian velocities is* $\sqrt{\frac{24}{24-2}} \approx 1.0445$ *. Is the empirical 1.0690 due to time dilation?*

Precession Scrub

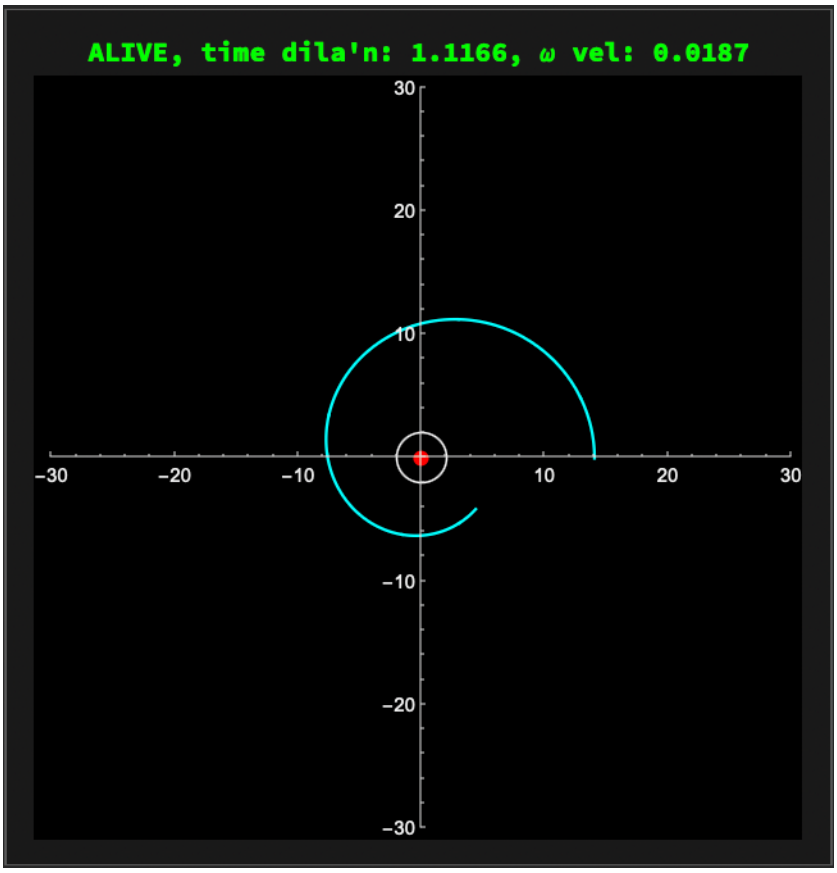
With the default parameters (press the RESET button), scrub the duration forward and backward slowly to watch the precession. Try the animation tools in the duration control at various speeds. The animation may be blocky at times, but if you pause the animation, the orbit eventually (and quickly) smooths out. This is a design feature of the animator: it prioritizes latency over fidelity just the same way your browser will sometimes give you a blocky image at first and then refine it progressively over time.

Delicate Escape

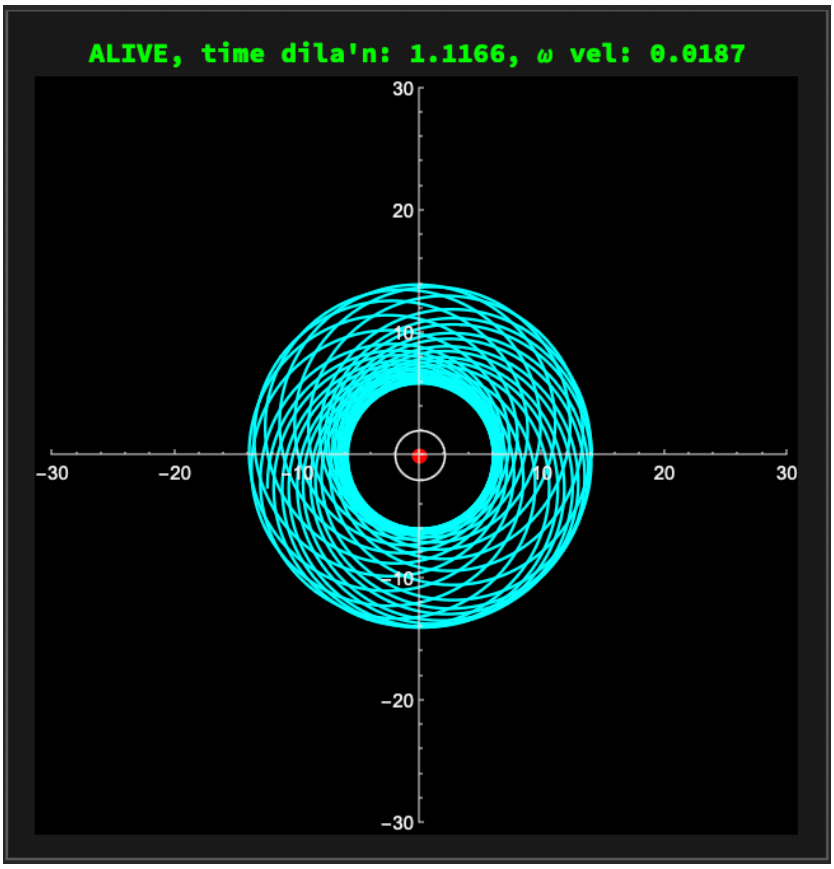
$r_0 = 14$, ecc = 0.95, duration = 130 → 135

At duration 130, you can see the “star” spiraling into the black hole. It doesn’t have enough ω to escape. By duration 135, it’s gone forever.

Now, open the Eccentricity control and set ecc = 0.98. The orbit is still alive at duration = 135.

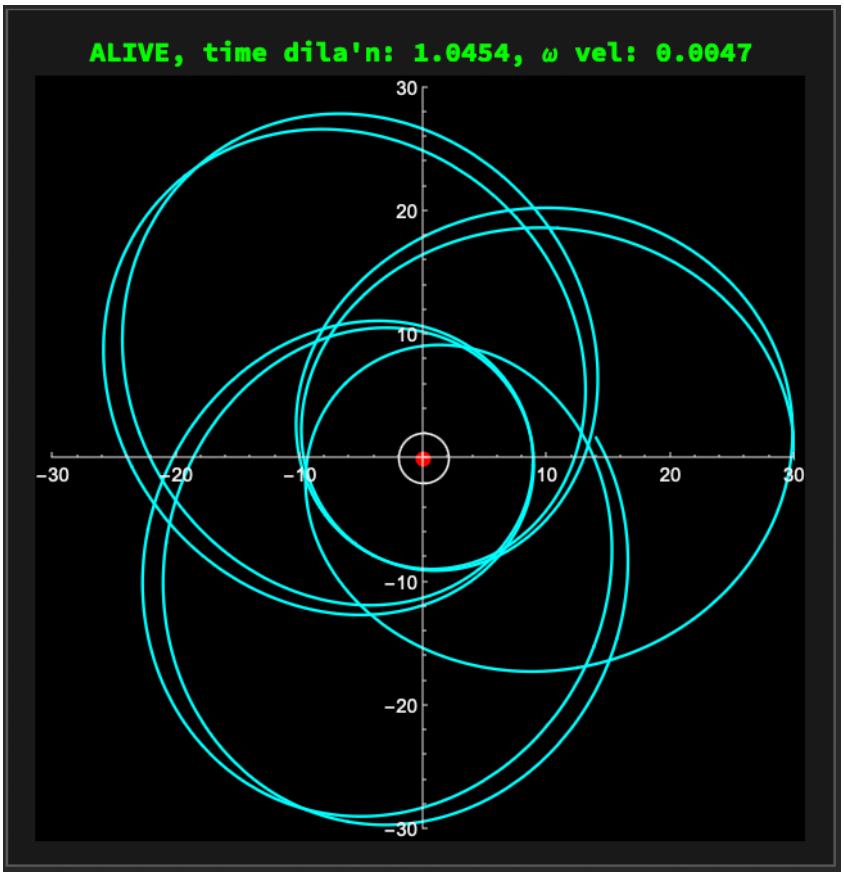


Increase the duration to the maximum (5000) and see the orbit surviving, probably forever.

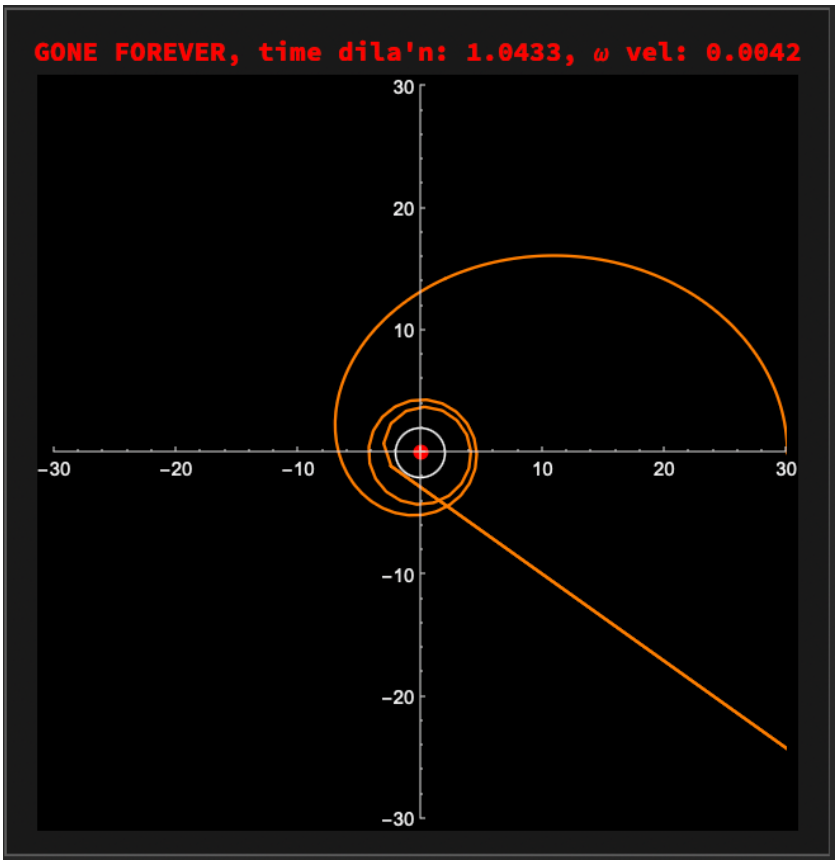


Nearly Periodic Trifoil Precession

Set r_0 = 30 (the max), eccentricity to 0.774, and “scrub” the duration right and left to watch precession trace out a three-leaf shape and very nearly repeat, but not quite.



Move the duration to max (5000), then slowly decrease the eccentricity until the black hole swallows the star at around ecc = 0.69. The precession pattern makes very interesting shapes along this journey to oblivion.



Play with the animation controls on the eccentricity, slowing it down by 12 clicks or so to watch the change from death-spiral to stable florets around ecc = 0.69.

Conclusion

In previous posts on the $\mathcal{UD}$ calculus, we focused on building the machinery: defining tensors, algebraic structures, and prototyping a compiler. In this post, we step back from just building the tool and use it for astrophysical science.

The best part of this experiment is what we *didn't* have to do. To simulate the extreme environment of a Schwarzschild black hole, where space-time curves, time slows, and orbits precess, we did not write new physics code. We simply fed the Schwarzschild metric into our existing $\mathcal{UD}$ compiler, cranked out the Christoffel symbols, and let the standard geodesic equation do the rest.

In the Smoke Test, we metaphorically reproduce the famous Precession of the Perihelion of Mercury around the Sun, just with a much heavier orbit-producing black hole. In the `Manipulate` demonstration, we can push parameters to probe the Innermost Stable Circular Orbit (ISCO) and visually confirm our test star vanishes from the Universe at the event horizon.

This exercise confirms that $\mathcal{UD}$ is solid, so far. It can serve as a foundation for exploring more complex geometries such as the Kerr metric, and ventured in future posts into quantum gauge theories.

For now, please enjoy the Interactive Lab. Crash some stars, search for resonances in precession, and

explore the fearsome math of general relativity.

References

1. Beckman, Brian. *From the steam age to quantum fields: a unified approach via UD tensorial calculus*. Wolfram Community post.
2. Beckman, Brian. *General relativity in the UD calculus*. Wolfram Community post.
3. Beckman, Brian. *Formal Differential Geometry in the UD Calculus: Part 1*. Wolfram Community post.
4. Beckman, Brian. *Covariant Derivatives and Connections in the $\mathcal{UD}$ Calculus*. Wolfram Community post.
5. Beckman, Brian. *Riemann Curvature in $\mathcal{UD}$ + Compiler POC*. Wolfram Community post.
6. Wolfram Function Repository. *MetricTensor*. For component-based calculations.
7. [MTW] Misner, Charles W.; Thorne, Kip S.; Wheeler, John Archibald. *Gravitation*. Princeton University Press, 2017.
8. Genzel, R., & Ghez, A., *Mass Measurement via S2 Orbit.* 2020 Nobel Prize in Physics.
9. Detection of Precession: GRAVITY Collaboration. (2020). “Detection of the Schwarzschild precession in the orbit of the star S2 near the Galactic centre massive black hole.” Astronomy & Astrophysics.
10. Andrew Ochadlick. Mass of Sagittarius A* from S2 orbital parameters. YouTube.